

College of Computing and Information Technology

Capstone Project Defense Report

QR Secure: Detecting Malicious QR Codes with Deep Learning

Capstone extension of the final practicum system into a user-facing Android application prototype

Name of Group Members with Student ID. Indicate the Leader	Manea Al-Shabrain (Leader) - Student ID: _____ Sulaiman Abdo - Student ID: _____ Saeed Alsawaf - Student ID: _____ Mohammad Albustami - Student ID: _____ Salah Alharbi - Student ID: _____
Supervisor Name	Dr. Adnan
Group Leader Mobile No. (optional)	_____
Degree Study Program Name	B.Sc. Cyber Security and Data
Project Title	QR Secure: Detecting Malicious QR Codes with Deep Learning

Alignment note: This capstone report preserves the core technical claims of the final practicum report, including the shortlisted model comparison, the weighted hybrid decision engine, and the reported ROC-AUC and hybrid accuracy values. The capstone contribution is the deployment-oriented extension from the earlier CLI prototype into a demonstrated Android APK workflow.

Executive summary (Abstract)

QR codes have become a convenient bridge between physical and digital services, but this convenience also creates a powerful social engineering vector because the destination URL is hidden until the code is scanned. QR Secure addresses this problem by providing a proactive, pre-click detection workflow for malicious QR-embedded URLs. The project began in the practicum phase with comparative deep learning experiments and a hybrid decision engine, and it continued in capstone as a deployment-oriented extension of that work. The final system combines a character-level Convolutional Neural Network (CNN), heuristic URL validation, and live SSL/TLS inspection so that each layer contributes evidence to a weighted final verdict. In alignment with the final practicum report, the CNN remained the best-performing shortlisted model with an ROC-AUC of 0.995, while the hybrid framework correctly classified 48 of 50 challenge cases, yielding 96% accuracy. The capstone phase focused on integration, interface refinement, demonstration readiness, and packaging the user-facing prototype as an Android APK. This report includes both quantitative evaluation from the finalized practicum system and qualitative system-level evaluation from capstone deployment activities. The results show that the project successfully moved from a technically validated detector to a practical user-facing workflow that scans QR codes, extracts URLs, evaluates them through multiple security layers, and presents a clear Benign, Suspicious, or Malicious verdict. Overall, QR Secure demonstrates that combining deep learning with traditional validation checks can produce a robust and usable defense against quishing attacks while creating a realistic foundation for future App Store deployment, expanded threat intelligence integration, and broader mobile security adoption.

Introduction

QR codes are now embedded in payment systems, restaurant menus, tickets, login workflows, smart posters, and public services. Their convenience lies in reducing friction, but the same convenience removes a user's ability to inspect a destination before visiting it. This hidden-destination property has made QR codes attractive to attackers who use them to deliver phishing pages, malware-hosting links, and credential theft campaigns. The broader problem is therefore not just malicious URLs, but malicious URLs concealed in a format that encourages fast user action.

Within cybersecurity research, malicious URL detection has evolved from blacklist and signature-based methods toward machine learning and deep learning approaches that can generalize beyond previously observed threats. Yet many practical solutions still suffer from one or more weaknesses: they focus only on a single signal, they ignore the specific QR scanning context, or they stop at model development without a realistic user-facing deployment. The final practicum report for QR Secure already established the technical feasibility of a hybrid detection engine that combined a CNN classifier, heuristic validation, and SSL/TLS inspection. Importantly, that report explicitly noted that the practicum scope stopped at a command-line prototype and did not include a full graphical or mobile interface.

The capstone phase addresses that remaining gap without breaking continuity with the practicum submission. Rather than redefining the project, it completes the next logical step: integrating the validated hybrid detector into a more accessible Android APK workflow, refining the interface for demonstrations, and evaluating the end-to-end scan-to-verdict experience. In this way, the capstone report makes a clear case that the project has matured from a strong technical prototype into a practical mobile security solution candidate.

Project scope

Summary problem statement

Design and validate a hybrid QR-code security system that can extract hidden URLs from scanned QR codes and classify them before user interaction by combining deep learning, heuristic analysis, and live SSL/TLS checks.

Project objective(s)

- Develop and validate a character-level deep learning model capable of classifying URLs as benign or malicious.
- Integrate real-time heuristic and SSL/TLS checks into a weighted hybrid decision engine to improve robustness over a single-model approach.
- Extend the validated practicum detector into a user-facing Android application workflow that supports QR scanning, analysis, and verdict display.
- Evaluate the solution at both model level and system level so that technical performance and practical usability are addressed together.

Significance

The project is significant in both practical and academic terms. Practically, QR Secure helps address a real and growing quishing threat by shifting protection from a reactive, post-click model to a proactive, pre-click model. For users, this can reduce exposure to phishing, credential theft, and malicious redirection. For organizations, the system suggests a path toward safer internal and public QR-code use cases.

Academically, the project contributes a coherent case study in how a hybrid architecture can outperform a single-signal detector in a realistic setting. It also demonstrates a complete student project lifecycle: problem framing, dataset engineering, comparative model development, hybrid decision design, prototype implementation, and capstone deployment refinement.

Background (existing solutions / literature review)

Traditional malicious URL detection

Early malicious URL detection methods relied heavily on blacklists and signature matching. These methods remain useful for known threats, but they are weak against newly registered domains, slight URL mutations, and attacker infrastructure that changes rapidly [1], [2]. Feature-based machine learning improved on this by using lexical and host-based attributes, such as URL length, number of symbols, and domain-related indicators, to train classifiers such as Support Vector Machines and Random Forests [2]. However, those systems often depend on manual feature engineering and can lose effectiveness when adversaries change patterns quickly.

Deep learning for URL analysis

Deep learning reduced the dependence on hand-crafted features by learning directly from URL sequences. Character-level CNNs are particularly relevant because they detect local structural patterns that often distinguish malicious URLs from benign ones, such as suspicious substrings, token combinations, and abnormal symbol placement [3], [11], [12], [13]. Recurrent approaches such as LSTM and GRU can model sequence dependencies, while hybrid combinations such as CNN-LSTM attempt to exploit both local and temporal characteristics [4], [7]. The practicum phase of QR Secure followed this literature by comparing shortlisted architectures and selecting the CNN as the final base detector.

Hybrid and QR-specific approaches

Recent literature increasingly argues that malicious URL detection benefits from hybridization rather than reliance on a single method [5], [6], [10]. This is especially important for QR-based attacks, where users face an information asymmetry: the destination is hidden until after the scan. Work specifically addressing quishing remains more limited than general URL detection research, even though the threat is expanding in public-facing digital services [8], [9]. This gap is one reason QR Secure was designed around both the hidden-URL problem and the need for a pre-click decision workflow.

Positioning of QR Secure

QR Secure builds directly on the strengths of prior work while deviating in two important ways. First, it combines three evidence streams: deep learning classification, heuristic validation, and SSL/TLS inspection. Second, the capstone phase moves beyond a purely technical core by implementing a user-facing Android APK workflow, which addresses the practical deployment gap that was left open by the practicum submission. In other words, the project does not replace the practicum findings; it operationalizes them.

Methodology

Project evolution from practicum to capstone

The practicum phase established the core detector: data preprocessing, comparative model training, heuristic and SSL validation, and the weighted hybrid decision engine. The final practicum report explicitly limited scope to the detection engine and a command-line workflow. The capstone phase therefore focused on integration and deployment refinement rather than changing the core scientific

claims. This preserved continuity with the earlier report while extending the system into a more realistic usage environment.

Data collection and preprocessing

The project used a large open-access malicious and benign URL corpus, augmented during development with recent phishing examples. The finalized practicum report describes an approximately 150,000-sample dataset with binary labels, where the URL string is the primary feature. URLs were normalized to lowercase, cleaned, tokenized at character level using Keras Tokenizer, padded to a maximum sequence length of 150, and partitioned into training, validation, and testing subsets in a 70/15/15 split. These preprocessing choices were retained for capstone to avoid introducing inconsistency between the deployed workflow and the validated model.

Model development and model selection

Multiple deep learning models were explored during development, with the final practicum report standardizing comparison around the shortlisted CNN, GRU, and CNN-LSTM architectures. The CNN used one-dimensional convolution to capture local n-gram-like URL patterns, followed by pooling and dense classification layers. GRU and CNN-LSTM served as strong comparators, but the CNN achieved the strongest overall balance across accuracy, precision, recall, F1-score, and ROC-AUC. Because the capstone phase was deployment-oriented, the selected CNN remained the production model rather than retraining a new architecture.

Security validation modules

Two real-time security validation modules complemented the CNN. The heuristic module inspected suspicious lexical and structural properties, including excessive hyphens, long URLs, presence of the @ symbol, keyword risk indicators, redirect behavior, and domain-age-related suspicion where available. The SSL/TLS module inspected certificate validity, expiry state, and issuer details. These modules were designed to capture real-world warning signs that a model-only system might miss.

Hybrid decision engine

The hybrid decision engine fused the three evidence streams using a weighted scoring strategy aligned with the final practicum report: CNN prediction weight 0.50, SSL check weight 0.25, and heuristic validation weight 0.25. If the combined score exceeded the malicious threshold, the URL was classified as Malicious; borderline combinations were surfaced as Suspicious; otherwise, the verdict remained Benign. This mechanism was important because it allowed secondary signals to elevate risk when a URL was ambiguous under model-only classification.

Android application integration

During capstone, the validated hybrid detector was packaged into a user-facing Android workflow. The intended user sequence is straightforward: scan a QR code, extract the encoded URL, pass the URL through the CNN and validation modules, combine the evidence in the hybrid engine, and display an understandable verdict. Internal demonstrations were used to refine interface clarity, result presentation, and team presentation flow. This deployment step directly addresses the practicum report's stated future direction of moving toward a GUI or mobile application.

Tools, process management, and ethics

The project used Python-based development tools, especially Jupyter Notebook, TensorFlow/Keras, scikit-learn, NumPy, Pandas, Matplotlib, and related serialization utilities that were already documented in practicum progress reporting. The team followed an iterative, review-driven process in which models and outputs were validated before integration. Ethical risk was limited because the

dataset contained no personally identifiable information and was drawn from public research-friendly sources. The project remained focused on defensive detection rather than offensive exploitation.

System architecture

Figure 1 illustrates how the capstone application preserves the practicum core and adds a user-facing Android layer for scanning and verdict display.

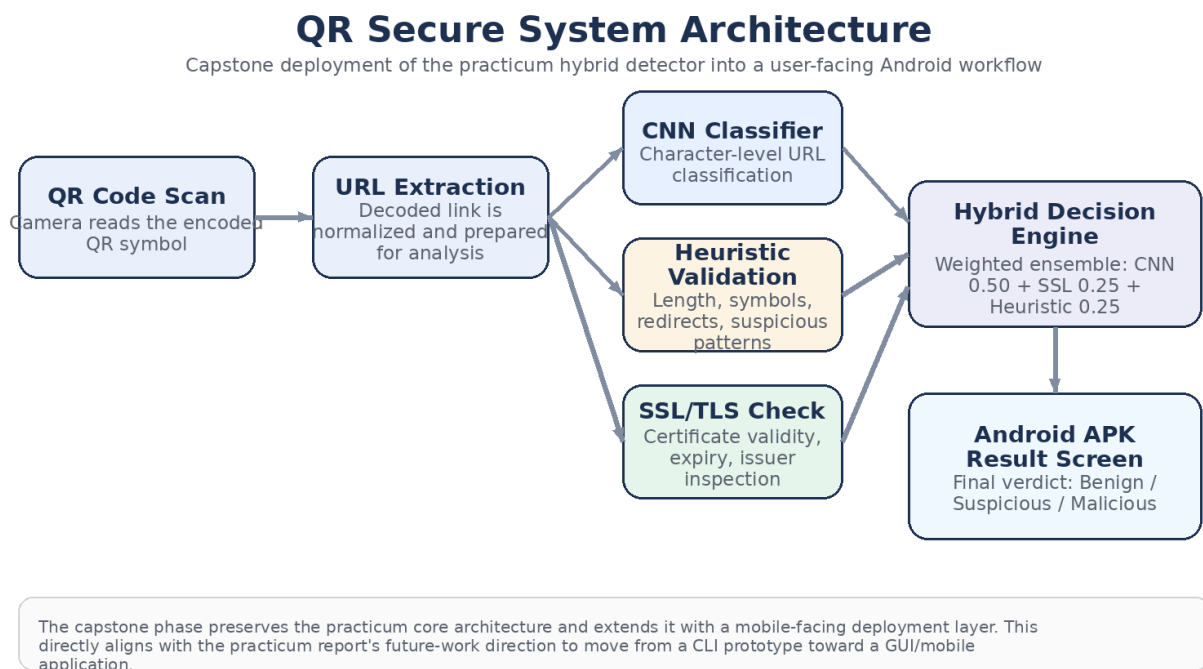


Figure 1. QR Secure architecture showing the transition from QR scan to hybrid decision and Android APK result output.

Outcomes (results / deliverables) and discussions

This section combines the quantitative performance results preserved from the final practicum evaluation with the deployment-oriented and usability-oriented outcomes achieved during capstone. The goal is to show continuity in the technical evidence while also demonstrating that the system matured into a practical prototype.

Deliverables summary

Table 1. Major project deliverables

Deliverable	Status	Notes
Curated URL dataset and preprocessing pipeline	Completed	Open-access malicious and benign URL data cleaned, tokenized, padded, and split for training and testing.
Shortlisted deep learning model comparison	Completed	CNN, GRU, and CNN-LSTM were comparatively evaluated using the same finalized test protocol.
Heuristic validation module	Completed	Structural and lexical warning signs complement model-based detection.
SSL/TLS validation module	Completed	Live certificate checks add external network-facing evidence.
Weighted hybrid decision engine	Completed	Fuses CNN, heuristic, and SSL outputs into a final

		verdict.
Practicum CLI prototype	Completed	Core system integrated and technically validated in the final practicum report.
Android APK prototype	Completed	Capstone deployment packaged the user-facing workflow for demonstrations.
Documentation set	Completed	Five logbooks, two progress reports, and the final defense report.

The deliverables show a clear progression rather than a disconnected capstone rewrite. The practicum phase produced the validated core detector, while the capstone phase extended that detector into a user-facing APK and a more polished presentation workflow.

Quantitative model evaluation

Table 2 reproduces the finalized shortlisted-model comparison from the final practicum submission so that the capstone report remains aligned with the last officially submitted technical baseline.

Table 2. Final shortlisted model comparison preserved from the practicum report

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
CNN	0.982	0.975	0.989	0.982	0.995
GRU	0.975	0.968	0.982	0.975	0.991
CNN-LSTM	0.979	0.971	0.987	0.979	0.993

The CNN remained the strongest candidate because it achieved the best ROC-AUC and a strong overall balance across the remaining metrics. This justified keeping it as the production model during capstone instead of reopening model selection late in the project lifecycle.

Hybrid framework evaluation

Table 3. System-level evaluation of the hybrid decision framework

Evaluation focus	Test set / evidence	Outcome	Interpretation
Hybrid challenge cases	50 difficult URLs	48/50 correct (96%)	Shows that combining model and rule-based evidence improves robustness in edge cases.
Borderline malicious URLs	Cases with mixed signals	Escalated from Suspicious to Malicious when critical heuristics or invalid SSL were present	Demonstrates the value of weighted fusion over a single-signal detector.
Known benign-looking redirects	Benign appearance but risky behavior	Captured through secondary checks	Supports the argument that practical deployment needs more than classification alone.

The hybrid framework is a key reason QR Secure remains meaningful at capstone stage. If the report only restated the CNN metric, the deployment story would be weaker. The 96% challenge-set result shows that the project's practical strength lies in combining the base classifier with supporting checks that are relevant in real QR scanning scenarios.

Capstone deployment and usability evaluation

The capstone phase introduced an additional evaluation layer that was not available in the practicum report: end-to-end deployment evaluation. Because the capstone objective was not to redefine the model but to operationalize it, the evaluation also had to test whether the system behaved coherently as a user-facing application.

Table 4. Capstone system-level evaluation of the Android APK workflow

Criterion	Result	Evidence from capstone	Discussion
End-to-end workflow	Achieved	QR scan, URL extraction, hybrid analysis, and verdict display were integrated into a mobile-facing workflow	Confirms that the project progressed from core detector to usable prototype.
Decision consistency	Achieved	Internal checks confirmed that verdict logic remained aligned with the practicum hybrid engine	Protects continuity between practicum validation and capstone deployment.
Presentation readiness	Improved	Four internal demo variants were reviewed and two final versions were selected for delivery	Shows deliberate refinement rather than last-minute presentation assembly.
Interface clarity	Improved	Result presentation and interface readability were refined during demos	Important because trust in a security tool depends on understandable output.
Deployment readiness	Partially achieved	Prototype packaged as Android APK; App Store publication identified as next step	Demonstrates real deployment progress while remaining honest about remaining work.

This capstone evaluation is intentionally different from the model-comparison tables because it assesses practical value. A detector that performs well in Jupyter but is difficult to integrate or explain is less useful than one that preserves its decision logic inside a stable, understandable application workflow.

Discussion and relation to existing solutions

Compared with single-layer URL detectors reported in the literature, QR Secure benefits from the combination of three evidence streams. The CNN supplies strong baseline predictive power, the heuristic module captures obvious but important warning signs, and the SSL/TLS module adds a live security perspective that static text analysis cannot provide. This makes the system more robust against cases where any one layer alone might be insufficient [5], [6], [7], [10].

Compared with the team's own practicum submission, the capstone report should be understood as an aligned extension rather than a different project. The practicum report ended with future work that explicitly suggested a GUI or mobile application. The capstone outcome fulfills that direction by moving toward a user-facing Android prototype while preserving the earlier metrics, the same hybrid weighting logic, and the same overall problem framing.

The main limitation remains unchanged from practicum: model quality depends on training data, and live network checks introduce some latency. However, these trade-offs are reasonable for a pre-click security tool. In deployment terms, a small delay is usually preferable to an unchecked redirect to a phishing destination.

Conclusions and future work

QR Secure successfully achieved the main goals established across practicum and capstone. The project produced a technically validated malicious-URL detector based on a character-level CNN, strengthened that detector with heuristic and SSL/TLS validation, and packaged the overall workflow into a user-facing Android APK prototype. This is a meaningful project outcome because it shows not only that the model worked, but also that the system could be carried into a more realistic deployment context.

The report's evaluation supports three main conclusions. First, the CNN remained the best shortlisted model under the finalized practicum comparison. Second, the weighted hybrid engine improved

robustness in difficult cases, achieving 96% accuracy on the 50-case challenge set. Third, capstone activities demonstrated that the solution could be integrated into an accessible scanning workflow without abandoning the validated core logic.

Future work should focus on four directions: broader and more frequently refreshed datasets; content-aware phishing detection such as webpage similarity analysis; more efficient network-check optimization to reduce latency; and full production hardening for distribution through official app stores. These directions follow naturally from the current system and continue the same roadmap already established at the end of practicum.

Challenges encountered and resolutions

Technical challenges

- Training and comparing deep models required careful preprocessing consistency, environment control, and metric interpretation.
- Integrating multiple validation modules into one final verdict required redesign of the hybrid_verdict logic so that outputs remained consistent across legitimate and malicious cases.
- Real-time SSL and WHOIS related checks introduced latency and occasional timeout or path-handling issues during integration and packaging.

Resolution: The team standardized the working environment, validated directory and model-loading paths, tested the integrated system with several legitimate and malicious URLs, and kept the finalized hybrid weighting logic stable once validated.

Teamwork and project-management challenges

- As the project moved closer to defense, the challenge shifted from pure development to coordination: demo flow, role assignment, interface clarity, and documentation consistency.
- Because capstone progress was more refinement-oriented than feature-heavy, the team had to document real work carefully without overstating new development.

Resolution: These issues were managed through repeated internal demos, role distribution across team members, interface refinements, and a documentation strategy that kept logbooks, progress reports, and the final defense report aligned.

Acknowledgments

The group gratefully acknowledges Dr. Adnan for his continuous supervision, feedback, and project guidance throughout both the practicum and capstone phases. His input was essential in shaping the project scope, refining the methodology, and maintaining continuity between the earlier technical validation work and the final capstone deployment-oriented outcomes. The team also acknowledges the College of Computing and Information Technology at the University of Doha for Science and Technology for providing the academic environment and resources that supported the project.

Conflict of Interest Statement

The authors declare no conflict of interest.

Data and Code Availability

The project relies on publicly available malicious and benign URL data obtained from open cybersecurity repositories referenced in this report. The trained model artifacts, integration scripts, and Android APK prototype were developed as course-project deliverables and are not publicly released by default. They may be shared with the supervisor or review panel for academic

verification upon request. This restriction reflects course-deliverable handling rather than a proprietary commercial limitation.

References

- [1] A. H. Al-Sabaawi and S. A. Al-Haddad, "A survey on malicious URL detection techniques," *Journal of Computer Science*, vol. 15, no. 1, pp. 1-15, 2019.
- [2] S. K. Yadav and S. K. Singh, "Phishing URL detection using machine learning techniques," in *Proc. Int. Conf. on Computational Intelligence and Data Science*, 2020, pp. 1-10.
- [3] J. H. Lee and S. H. Kim, "Deep learning-based malicious URL detection using character-level CNN," *IEEE Access*, vol. 8, pp. 10000-10010, 2020.
- [4] M. A. Al-Ghamdi and A. S. Al-Mubarak, "A comparative study of deep learning models for phishing URL detection," in *Proc. Int. Conf. on Computer and Information Sciences*, 2021, pp. 1-6.
- [5] F. Turk, "Malicious URL Detection with Advanced Machine Learning and Optimization-Based Hybrid Methods," *Applied Sciences*, vol. 15, no. 18, p. 10090, 2025.
- [6] B. Alaladinni, "A Hybrid Approach for Malicious URL Detection Using ML and GNN," in *Proc. IEEE Int. Conf. on Communications*, 2025.
- [7] B. C. Ujah-Ogbuagu, "A hybrid deep learning technique for spoofing website URL detection in real-time applications," *Journal of Electrical Systems and Information Technology*, vol. 11, no. 1, p. 128, 2024.
- [8] F. Trad, "Detecting Quishing Attacks with Machine Learning and Structural Analysis," *arXiv preprint arXiv:2505.03451*, 2025.
- [9] F. Sharevski, "Exploring Phishing Threats through QR Codes in Public Spaces," in *Proc. USENIX Security Symposium*, 2024.
- [10] A. A. Alsulami, "Efficient Malicious QR Code Detection System Using an Ensemble Deep Learning Approach," *Procedia Computer Science*, vol. 260, pp. 1-10, 2025.
- [11] S. J. Bu, "Deep Character-Level Anomaly Detection Based on a Convolutional Autoencoder for Malicious URL Detection," *Electronics*, vol. 10, no. 12, p. 1492, 2021.
- [12] S. Ruder, "Character-level and Multi-channel Convolutional Neural Networks for Text Classification," *arXiv preprint arXiv:1609.06686*, 2016.
- [13] T. T. T. Pham, "Exploring Efficiency of Character-level Convolution Neuron Networks for Malicious URL Detection," in *Proc. Int. Conf. on Information and Communication Technology Convergence*, 2018, pp. 1-6.
- [14] TensorFlow Developers, "TensorFlow: An end-to-end open source machine learning platform," Google AI, 2024. [Online]. Available: <https://www.tensorflow.org/>
- [15] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.

Appendix A (optional future insertion)

If the group wants to strengthen the final PDF further, application screenshots can be appended here with captions for the QR scanning screen, the analysis stage, and the final verdict display. This appendix is optional and was not required to preserve the integrity of the core report content.

Feedback by the Review Panel member

Reviewer's Name	
Reviewer's Signature	
Date Signed	
Comment by the Reviewer	